

Quick Guide: From Idea to Software

1. Understand the purpose

Before thinking about features or technology:

What problem are we solving?

Define:

- Who has the problem?
 - What are they trying to accomplish?
 - What should be different when the process is complete?
-

2. Map the information flow

Use this universal sequence:

**Person/System → Capture → Data → Process → Decision → State Change
→ Output → API/Event → Next Component**

Stage	Ask
Person / System	Who or what initiates this?
Capture	How does information enter?
Data	What information is required?
Process	What needs to happen to it?
Decision	What rules or conditions apply?
State Change	What changes as a result?
Output	What is produced?
API / Event	What needs to be exposed or communicated?
Next Component	What uses the result next?

The flow should be understandable **without seeing any UI or code**.

3. Identify the data

Determine:

What to capture → What to store → What to calculate/process → What to retrieve → What to pass onward

Also establish:

- unique identifiers
- relationships
- ownership
- validation
- required vs optional data
- history/audit requirements
- retention/deletion requirements

The goal is to create **one consistent version of reality**.

4. Break the problem into small capabilities

Don't immediately build a large application.

Find the smallest sensible reusable functions.

For example, universally:

Large System

- Authentication
- File Upload
- Image Upload
- Search
- Calculator
- Task
- Reminder
- Notification
- Approval
- Document
- Audit

Each component should have a **clear responsibility**.

5. Define every component

Every component should answer:

PURPOSE



INPUT



PROCESS



RULES



STATE



OUTPUT



API / EVENT

If this cannot be explained simply, the component may be doing too much.

6. Keep components independent

A component should know **only what it needs to perform its job**.

Avoid making one component responsible for:

A



B



C



D



E

when these can reasonably exist as:

[A] [B] [C] [D] [E]

↓ ↓ ↓ ↓ ↓

Clear inputs and outputs

This makes components easier to build, test, replace and reuse.

7. Define the connections

Now establish how the pieces communicate.

For every connection:

COMPONENT A

↓

What does it send?

↓

In what format?

↓

When?

↓

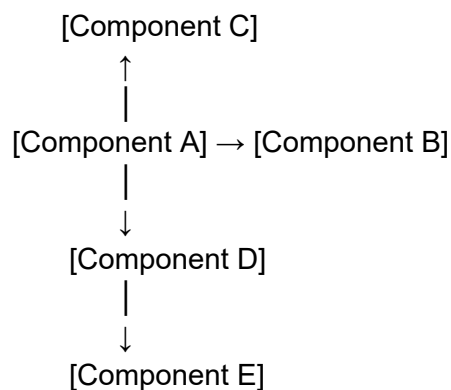
COMPONENT B

Define the **contract**, not just the connection.

Component B should know exactly what it can expect from Component A.

8. Assemble the jigsaw

Only after individual pieces are understood should larger workflows be assembled.

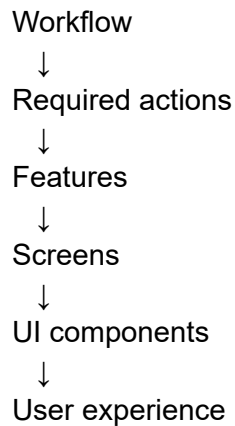


The individual components remain relatively simple.

Complex behaviour emerges from how the components are connected.

9. Add features and interface

Now determine what humans need to interact with the system:

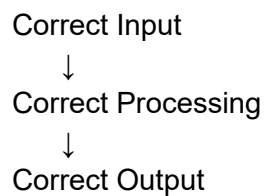


The interface should represent the underlying workflow — **not define the architecture.**

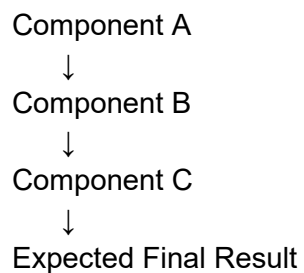
10. Build and test

Test at two levels.

Component level



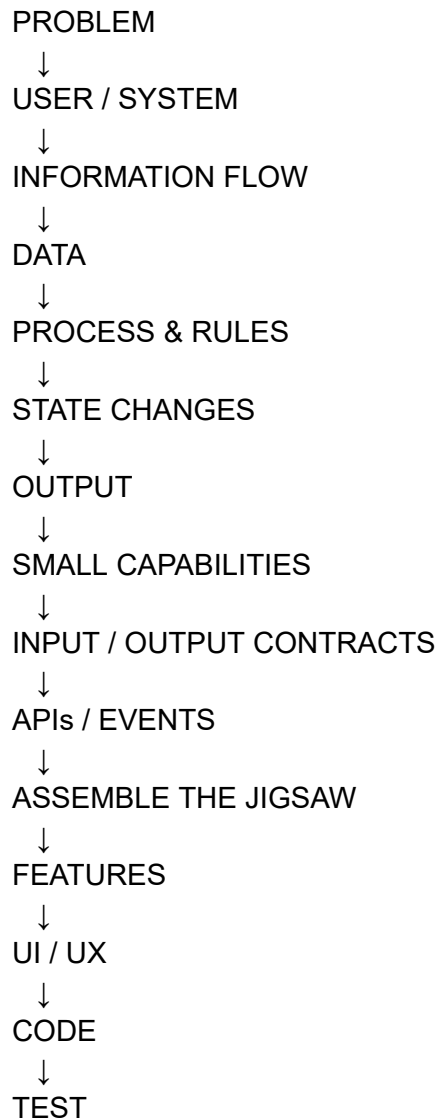
System level



A component working correctly does not automatically mean the overall workflow works correctly.

Universal Development Sequence

The whole guide can ultimately be reduced to this:



Three rules to remember

- 1. Understand the information before designing the application.**
- 2. Build small pieces with clear inputs, responsibilities and outputs.**
- 3. Assemble complexity from simple pieces instead of putting complexity inside every piece.**

That is universal enough to apply to a calculator, accounting system, marketplace, CRM, hospital system, social network, ERP, mobile app, or almost any other software product.

